GEMINI CLI:
A3 : Architectural Enhancement

Group Name : Group 25

CISC 322/326

Date : April 6th , 2026

Video Link : https://youtu.be/OuZeUjFKsAE

Group Names & Roles

Caravaggio, Emanuel - Presenter , Introduction & Overview, Conclusion, AI Collaboration , (Student I.D : 20403106)

Clulow, Damon - Abstract, Impact on Conceptual Architecture, Potential Risks (Student I.D : 20285634)

Dhungana, Raghab - SAAM Analysis, Impact on System Qualities (Student I.D : 20354596)

Elamin, Adam - Use Case 2, Final Formatting (Student I.D : 20271708)

Fernando, Paul - Team Leader, New Feature Overview, Use Case 1, Lessons Learnt (Student I.D : 20261647)

Shetty, Aditya - Presenter , Implementation Approaches, Testing (Student I.D : 20320373)

Table Of Contents

Abstract - 4

Introduction and Overview - - - - - 5

New-Features Overview - - - - - - - - 6

Implementation Approaches - - - - 7

Approach #1 - - - - - - - - - - - - - - - - - - 8

Approach #2 - - - - - - - - - - - - - - - - - - 9

SAAM Analysis - - - - - - - - - - - - - - 10

Impact On Conceptual Architecture - - - - - - - - - - - 11

Impact on System Qualities - - - - - - - - - - - - - - - - - - 12

Potential Risks - 13

Use Case #1 - 14

Use Case #2 - 15

Testing - 16

Lessons Learned - 17

Conclusions - 18

AI Collaboration - 19

Abstract

- Gemini CLI safety relies on identifying dangerous commands and prompting the user for approval, which can at times be usurped

- We propose Stateful Environment Simulation (SES) Engine, which adds a simulation layer where commands are tested against a snapshot of the user's file system before executing

- We covered Two Implementations via SAAM analysis, with the containerized sandbox chosen over the virtual overlay for its isolated execution environment.

- This enhancement improves safety and user experience while adding an extra layer of depth to the GEMINI environment

Introductions and Overview

- The SES Engine addresses the CLI's difficulty with predicting real-world effects of dangerous commands before execution, shifting the system from a reactive to a proactive safety model.
- We integrated the Engine into the Core Layer, interacting with the orchestration engine, security, and tool execution framework, which classifies and targets environment-modifying operations.
- Two implementations we investigated, include a virtual overlay and a containerized sandbox, evaluated through SAAM analysis against NFR's.
- This enhancement makes the Gemini CLI a safer and more transparent system, improving reliability and user trust when executing complex AI-driven tasks.

New Features Overview

- The SES Engine "dry-runs" the execution context, allowing dangerous commands to be simulated against a snapshot of the user's file system before committing to the user's environment

- The engine runs in the Core Layer as a direct dependency of the Agentic Reasoning subcomponent, sitting between the Policy module and Tool Execution Framework.

- The SES Engine has a Pipe-and-Filter style architectural style, comprised of five subcomponents: The Simulation Controller, State Snapshotter, Virtual Overlay, Outcome Analyzer, and Integrity Validator.

- The engine activates only for environment-modifying operations such as sudden file or system configuration changes and process execution, avoiding unnecessary overhead for read-only tasks.

Implementation Approaches

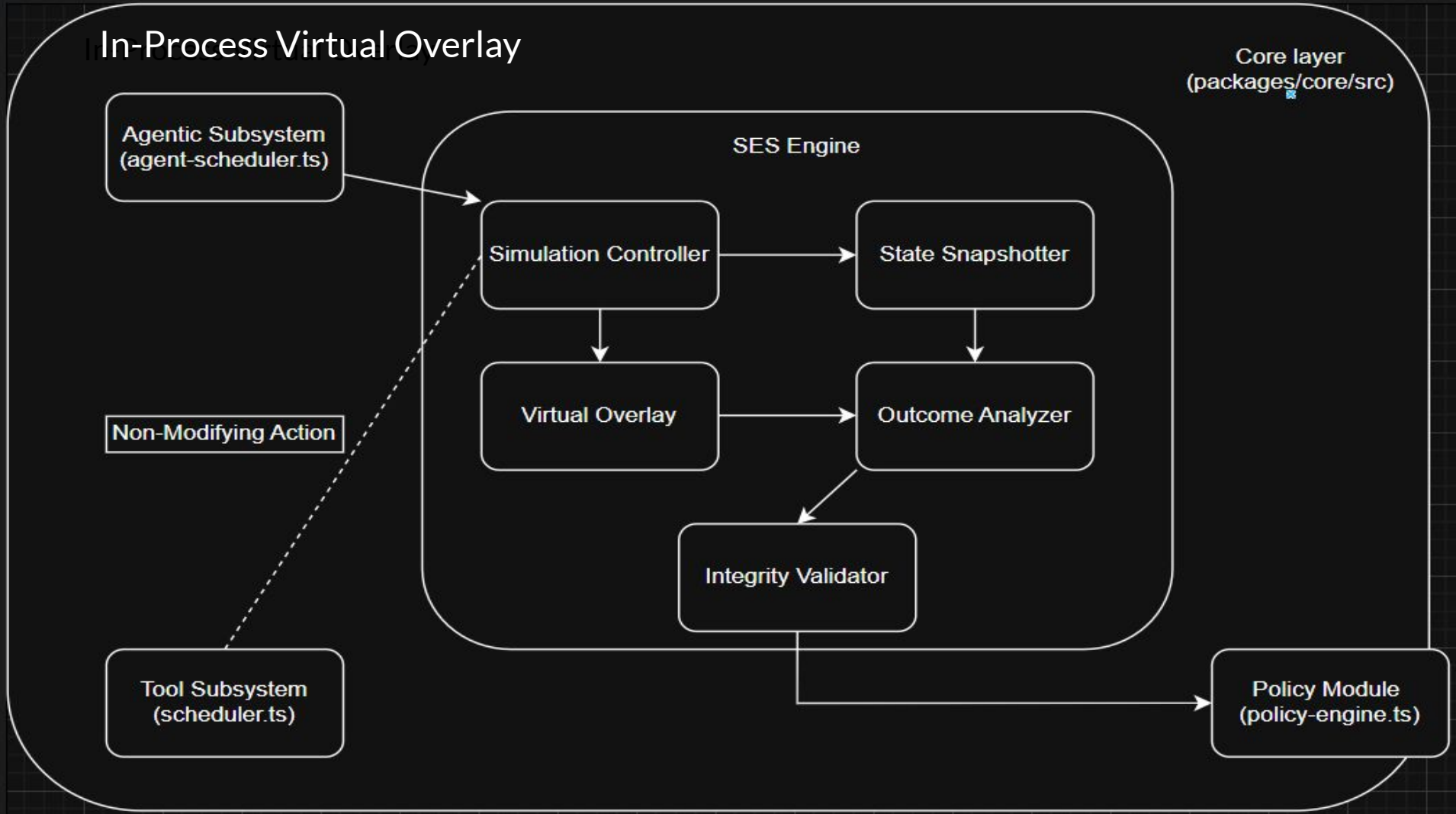
- Two implementation approaches are an in-process virtual overlay and a containerized sandbox, both activated only for environment-modifying operations.

- The virtual overlay intercepts Node.js file system calls at runtime, building an in-memory snapshot without copying files, however this makes it unable to intercept binaries bypass Node.js APIs.

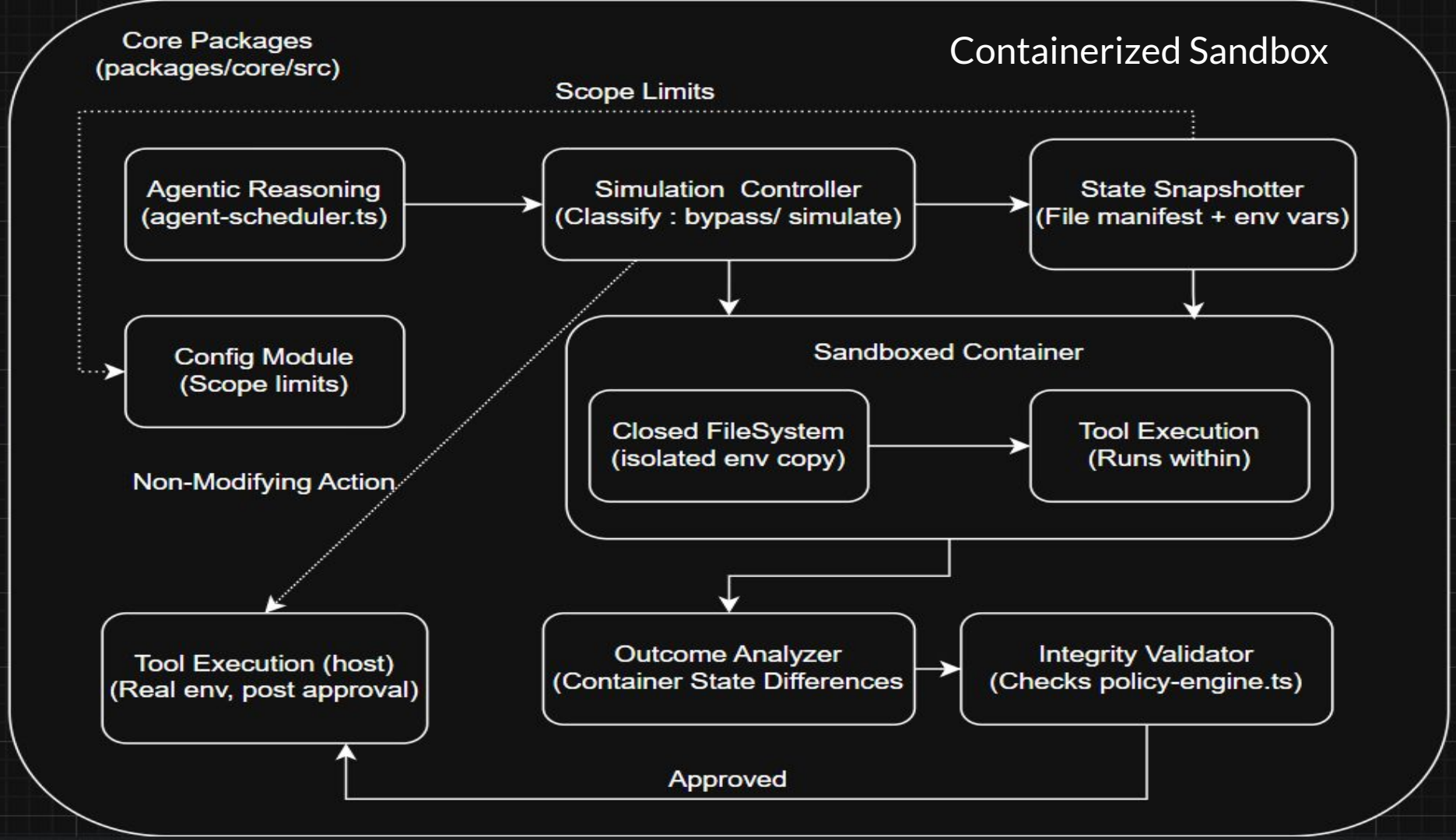
- The containerized sandbox copies files into a Docker instance where it runs entirely contained within that instance, capturing all side effects, introducing slower startup times and an additional runtime dependency.

- Both approaches follow the same format through the Simulation Controller, State Snapshotter, Outcome Analyzer, and Integrity Validator before presenting results to the user.

In-Process Virtual Overlay



Containerized Sandbox



SAAM Analysis

- **Stakeholders & Goals:** Developers (portability, maintainability) vs. Users (safety, performance)
- **Approach 1 (Virtual Overlay):** Fast & portable, but weaker safety and high maintenance
- **Approach 2 (Container Sandbox):** Strong security & easier maintenance, but slower performance
- **Final Decision:** Choose Container Sandbox since it prioritizes **safety and reliability** despite added latency

Impact On Conceptual Architecture

- Adds SES Engine to Service Layer; modifies request flow to *simulate before execution* for risky actions

- Increased responsibilities + new interactions across components; mitigated through modular design and clear interfaces
  - Orchestration, Security, Terminal, and Tool Execution now integrate with SES Engine

- **Low-Level Design:** Pipe-and-filter SES Engine
  - State Snapshooter, Virtual Overlay, Outcome Analyzer, and Integrity Validator

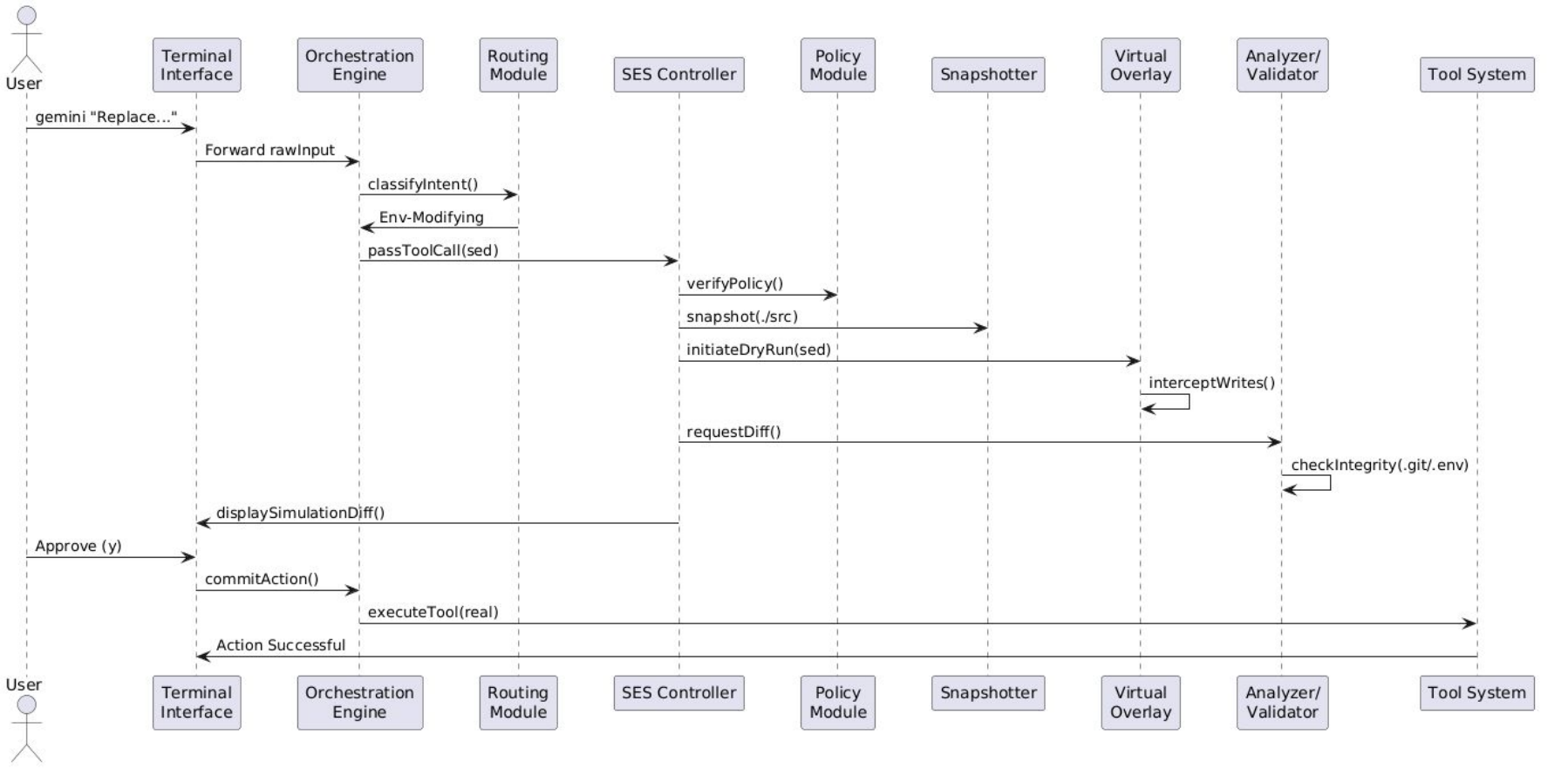
Impact on System Qualities

- **Security & Safety:** Proactive simulation prevents harmful AI actions before execution
- **Performance:** Added latency from simulation pipeline (snapshot + analysis)
- **Usability Tradeoff:** More user steps (approval required) but safer interactions
- **Transparency & Trust:** Users see predicted outcomes, making AI behavior more understandable and reliable

Potential Risks

- Added SES Engine increases architectural complexity and inter-component interactions
  - mitigate with modular design & clear interfaces
- Simulation and validation introduce latency
  - mitigate with configurable execution and resource controls
- Simulated results may differ from real outcomes
  - improve accuracy and continuously validate

Use Case #1 - Validating a Batch File Refactor



Use Case #2 -Validating Shell Configuration and Environment Integrity

1. Input: The user requests a performance optimization for their shell environment.
2. Classification: The Orchestration Engine flags intent as a high-risk operation.
3. Simulation Trigger: A dry-run is mandated due to a protected system file (.bashrc).
4. Simulation Execution: The current shell state is snapshotted; proposed changes execute in the Virtual Overlay.
5. Safety & Integrity: A diff is generated; the Integrity Validator flags a PATH modification which risks prioritizing local directories over system binaries.
6. User Review: User sees the exact line changes and receives a PATH security warning.
7. Commitment: Upon approval, changes are applied to the host environment.

Testing

- **Unit Testing:** Test each SES subcomponent independently (Controller, Snapshotter, Overlay, Analyzer, Validator)
  - verify correct routing, accurate snapshots, safe simulations, correct outcome analysis, and integrity checks
- **Integration Testing:** Ensure SES Engine interacts correctly with Orchestration Engine, Security & Execution, and Tool Execution Framework

- **End-to-End Validation:** Full simulation from user request to execution
  - confirm correct differences, user approval flow, and environment safety; check performance and latency under different conditions

- **Performance Checks:** Measure latency for both approaches; confirm non-modifying operations remain fast and unaffected

Lessons Learned

- Learned that true safety requires predicting command effects, not just analyzing text prompts
- In-Process Overlay = fast but risky; Container Sandbox = secure but slower
- Importance of preserving Core Layer independence and one-way dependencies while adding SES Engine

Conclusions

- SES Engine enables outcome-based command simulation, reducing risk of destructive actions
- Clear previews of command outcomes let users make informed decisions
- Adds complexity and responsibilities, but strengthens Gemini CLI’s safety, usability, and robustness

AI Collaboration

- used for clarifying SES Engine descriptions, comparing approaches, and analyzing stakeholder concerns
- focused on providing sufficient context and restricting the models responses to the information we supplied
- To ensure accuracy: report content; outputs reviewed, validated, and cross-referenced to ensure alignment with design and course concepts
- AI contributed ~10% by refining explanations, aiding discussion;
- effective use requires clear tasks, careful prompting, and output validation

Gemini